



Medical Laboratory System

Database Design & Implementation Report

Under Supervision Of: Mohamed Mahmoud Elsayeh

Name	ID
Malak Ali Allam	241001107
Malak Ashraf Ghoneim	241001356
Saria Mohamed Abdelmoniem	241002118
Habiba Ahmed Fareed	241000534

Table of Contents

1. Conceptual Design – Entities & Attributes.....	2
2. ER Diagram (Conceptual)	4
3. Relational Schema (Mapping / Table Diagram).....	5
4. MySQL Workbench Physical Schema	6
5. DDL – CREATE TABLE Statements.....	7
6. DML – INSERT Statements & Test Data	12
7. Views & Queries	20
8. Index.....	21
9. Trigger	22
10. Design Rationale & Implementation Notes.....	23
11. Application / UI Source Code.....	25

1. Conceptual Design – Entities & Attributes

The Medical Laboratory System models core clinical laboratory operations: tracking patients, the medical tests they undergo, the laboratorians who perform those tests, the physical components consumed by each test, and the recorded test results. Nine entities are identified.

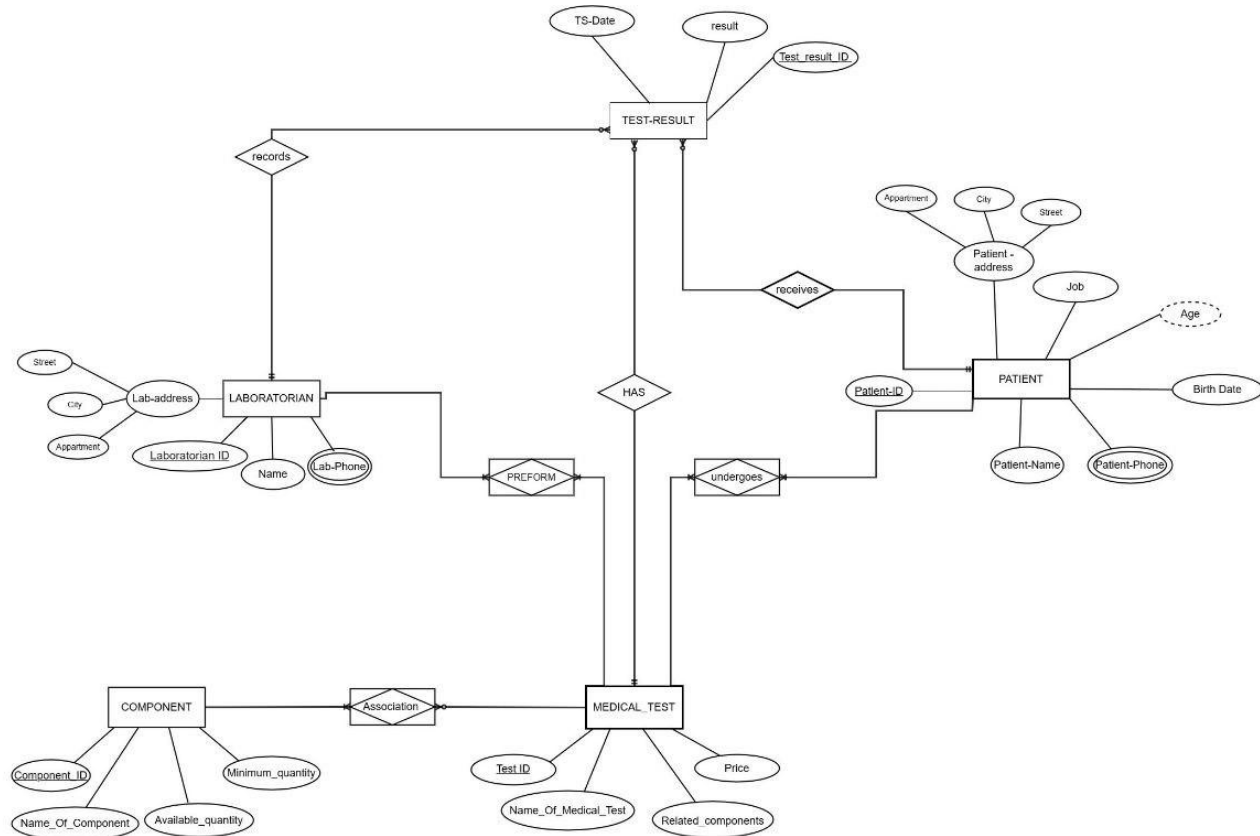
Entity	Attributes	Primary Key	Notes
LABORATORIAN	Laboratorian_ID, Name, Lab-address (Street, City, Apartment), Lab-Phone (multi-valued)	Laboratorian_ID	Lab-Phone extracted to own table
PATIENT	Patient_ID, Patient-Name, Job, Birth_Date, Age (derived), Patient-address, Patient-Phone	Patient_ID	Age derived from Birth_Date
MEDICAL_TEST	Test_ID, Name_Of_Medical_Test, Related_components, Price, Laboratorian_ID (FK)	Test_ID	References Laboratorian
COMPONENT	Component_ID, Name_Of_Component, Available_quantity, Minimum_quantity	Component_ID	Inventory tracking
ASSOCIATION	Component_ID (FK), Test_ID (FK)	(Component_ID, Test_ID)	Resolves M:N COMPONENT–MEDICAL_TEST
TEST_RESULT	Test_Result_ID, Result, TS_Date, Laboratorian_ID (FK), Patient_ID (FK), Test_ID (FK)	Test_Result_ID	Central fact table
PREFORM	Laboratorian_ID (FK), Test_ID (FK)	(Laboratorian_ID, Test_ID)	Resolves M:N Laboratorian–MedicalTest
UNDERGOES	Patient_ID (FK), Test_ID (FK)	—	Resolves M:N Patient–MedicalTest
LAB_PHONE	Laboratorian_ID (FK), Lab_phone	(Laboratorian_ID, Lab_phone)	Multi-valued attr of Laboratorian
PATIENT_PHONE	Patient_ID (FK), Patient_Phone	(Patient_Phone, Patient_ID)	Multi-valued attr of patient

Relationships Summary

Relationship	Entities	Cardinality	Participation
records	LABORATORIAN – TEST_RESULT	1: N	Total on TEST_RESULT side
receives	PATIENT – TEST_RESULT	1: N	Total on TEST_RESULT side
HAS	TEST_RESULT – MEDICAL_TEST	N: 1	Total on TEST_RESULT side
PREFORM	LABORATORIAN – MEDICAL_TEST	M: N	Partial both sides
undergoes	PATIENT – MEDICAL_TEST	M: N	Partial both sides
Association	COMPONENT – MEDICAL_TEST	M: N	Partial both sides
has phone	LABORATORIAN – LAB_PHONE	1: N	Partial on Laboratorian
has phone	PATIENT – PATIENT_PHONE	1: N	Partial on Patient

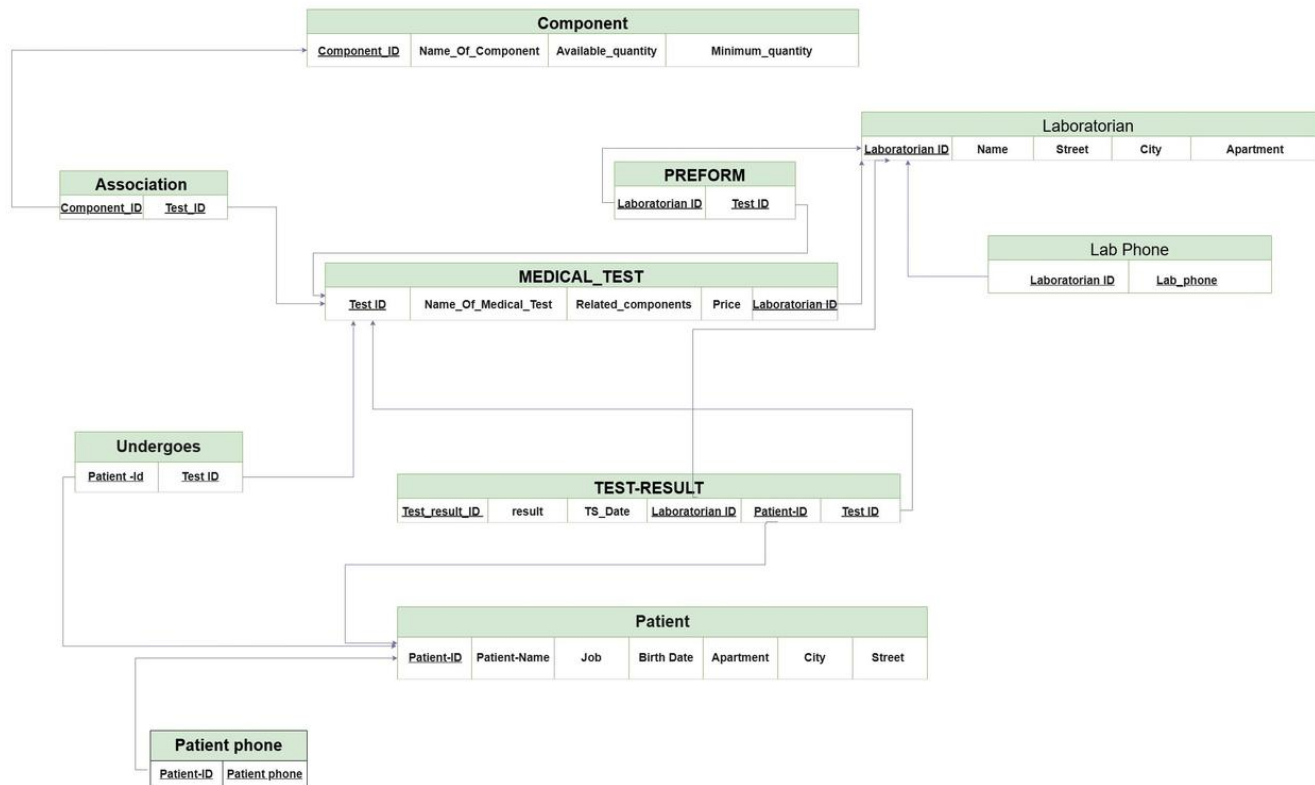
2. ER Diagram (Conceptual)

The ER diagram below uses standard Chen notation: rectangles for entities, ellipses for attributes (dashed ellipse for derived attribute Age), diamonds for relationships, and double lines for total participation. Cardinality ratios are annotated on each relationship line.



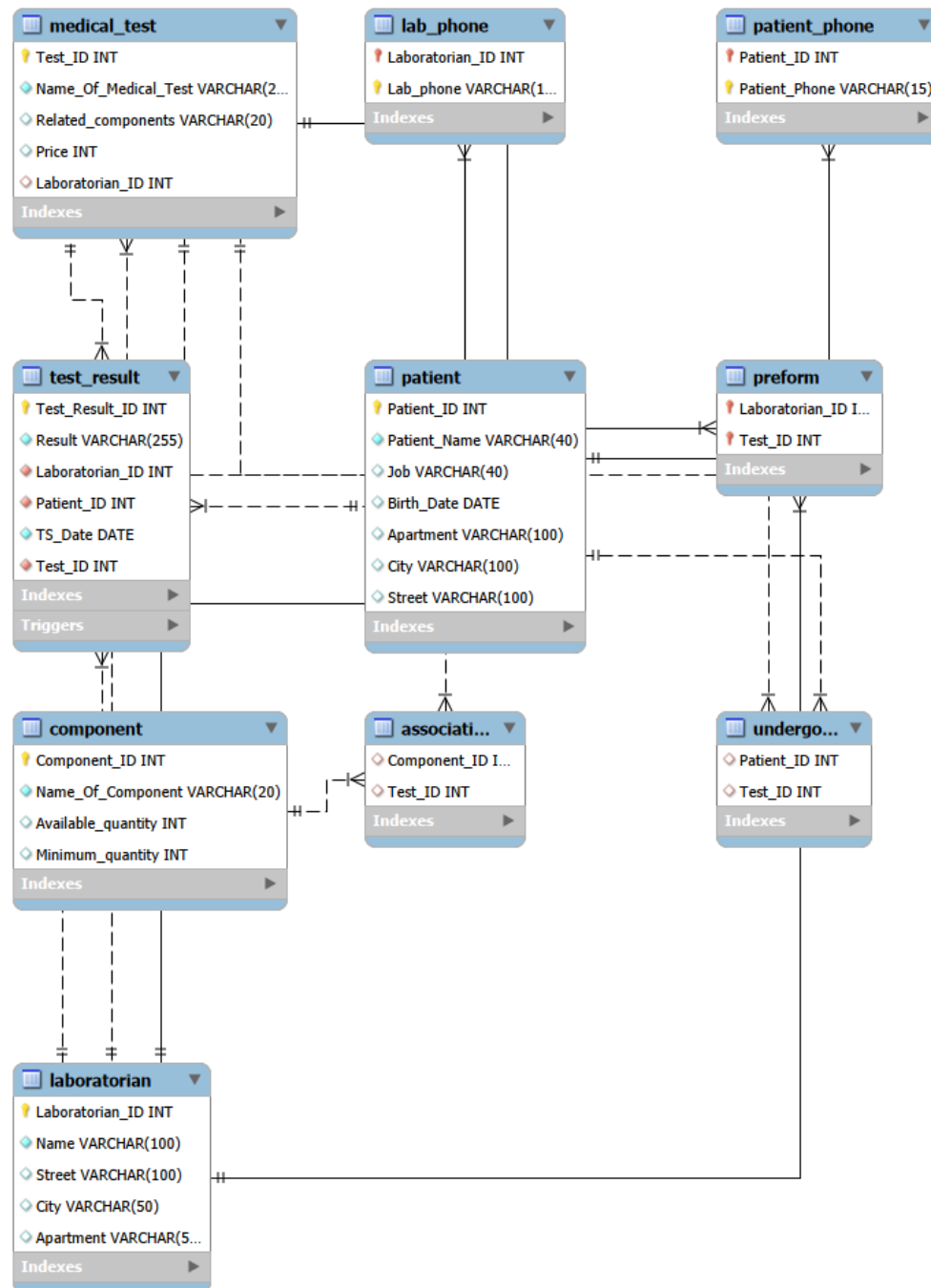
3. Relational Schema (Mapping / Table Diagram)

The mapping diagram translates the ER model into relational tables. Each table is shown with its attribute columns; underlined attributes denote primary keys, and arrow connectors represent foreign-key relationships.



4. MySQL Workbench Physical Schema

The screenshot below is the physical schema generated by MySQL Workbench, reflecting the actual tables created by the DDL. It shows data types, primary keys (gold key icon), foreign keys (red diamond), and all relationship lines with crow-foot notation.



5. DDL – CREATE TABLE Statements

The following DDL creates the PROJECT database and all tables with primary-key, foreign-key, NOT NULL, AUTO_INCREMENT, ON DELETE, and ON UPDATE constraints. ALTER TABLE statements add circular foreign-key references after all tables have been created.

Database & COMPONENT

```
1 • drop database PROJECT;
2 • create database PROJECT;
3 • use PROJECT;
4
5 • create table if not exists COMPONENT (
6     Component_ID int,
7     Name_Of_Component varchar(20) not null,
8     Available_quantity int,
9     Minimum_quantity int,
10    primary key (Component_ID)
11 );
```

MEDICAL_TEST

```
13 • create table if not exists MEDICAL_TEST (
14     Test_ID int,
15     Name_Of_Medical_Test varchar(20) not null,
16     Related_components varchar(20),
17     Price int,
18     Laboratorian_ID int,
19     primary key (Test_ID)
20 );
```

ASSOCIATION (junction: COMPONENT -- MEDICAL_TEST)

```
22 • create table if not exists ASSOCIATION (
23     Component_ID int,
24     Test_ID int
25 );
```


PREFORM (junction: LABORATORIAN -- MEDICAL_TEST)

```
27 • ○ REATE TABLE IF NOT EXISTS PREFORM (  
28     Laboratorian_ID int NOT NULL,  
29     Test_ID int NOT NULL,  
30     PRIMARY KEY (Laboratorian_ID, Test_ID),  
31     FOREIGN KEY (Test_ID) REFERENCES MEDICAL_TEST(Test_ID)  
32         ON DELETE RESTRICT  
33         ON UPDATE CASCADE  
34     ;
```

LABORATORIAN

```
36 • ○ REATE TABLE Laboratorian (  
37     Laboratorian_ID int PRIMARY KEY,  
38     Name varchar(100) NOT NULL,  
39     Street varchar(100),  
40     City varchar(50),  
41     Apartment varchar(50)  
42     ;
```

LAB_PHONE

```
44 • ○ CREATE TABLE Lab_Phone (  
45     Laboratorian_ID int,  
46     Lab_phone varchar(15),  
47     PRIMARY KEY (Laboratorian_ID, Lab_phone),  
48     FOREIGN KEY (Laboratorian_ID) REFERENCES Laboratorian(Laboratorian_ID)  
49         ON DELETE CASCADE  
50         ON UPDATE CASCADE  
51     );
```

PATIENT

```
53 • ○ create table if not exists Patient (  
54     Patient_ID int,  
55     Patient_Name varchar(40) not null,  
56     Job varchar(40),  
57     Birth_Date date,  
58     Apartment varchar(100),  
59     City varchar(100),  
60     Street varchar(100),  
61     primary key (Patient_ID)  
62 );
```

PATIENT_PHONE

```
64 • ○ create table if not exists Patient_Phone(  
65     Patient_ID int,  
66     Patient_Phone varchar(15),  
67     primary key (Patient_Phone,Patient_ID),  
68     foreign key (Patient_ID) REFERENCES Patient(Patient_ID)  
69         ON DELETE CASCADE  
70         ON UPDATE CASCADE  
71 );
```

TEST_RESULT

```

73 • CREATE TABLE IF NOT EXISTS TEST_RESULT (
74     Test_Result_ID int NOT NULL AUTO_INCREMENT,
75     Result varchar(255) NOT NULL,
76     Laboratorian_ID int NOT NULL,
77     Patient_ID int NOT NULL,
78     TS_Date date NOT NULL,
79     Test_ID int NOT NULL,
80     PRIMARY KEY (Test_Result_ID),
81     FOREIGN KEY (Laboratorian_ID) REFERENCES Laboratorian(Laboratorian_ID)
82         ON DELETE RESTRICT
83         ON UPDATE CASCADE,
84     FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID)
85         ON DELETE RESTRICT
86         ON UPDATE CASCADE,
87     FOREIGN KEY (Test_ID) REFERENCES MEDICAL_TEST(Test_ID)
88         ON DELETE RESTRICT
89         ON UPDATE CASCADE
90 );

```

UNDERGOES (junction: PATIENT -- MEDICAL_TEST)

```

92 • CREATE TABLE IF NOT EXISTS undergoes (
93     Patient_ID int,
94     Test_ID int,
95     foreign key (Patient_ID) REFERENCES Patient(Patient_ID),
96     foreign key (Test_ID) REFERENCES MEDICAL_TEST(Test_ID)
97 );

```

ALTER TABLE – deferred FK references

```

99 • ALTER TABLE MEDICAL_TEST
100     ADD FOREIGN KEY (Laboratorian_ID)
101     REFERENCES Laboratorian(Laboratorian_ID)
102     ON DELETE SET NULL
103     ON UPDATE CASCADE;
104
105 • ALTER TABLE ASSOCIATION
106     ADD FOREIGN KEY (Component_ID)
107     REFERENCES COMPONENT(Component_ID)
108     ON DELETE SET NULL
109     ON UPDATE CASCADE;
110
111 • ALTER TABLE ASSOCIATION
112     ADD FOREIGN KEY (Test_ID)
113     REFERENCES MEDICAL_TEST(Test_ID)
114     ON DELETE SET NULL
115     ON UPDATE CASCADE;

```

```
117 • ALTER TABLE PREFORM
118     ADD FOREIGN KEY (Laboratorian_ID)
119     REFERENCES Laboratorian(Laboratorian_ID)
120     ON DELETE RESTRICT
121     ON UPDATE CASCADE;
```

6. DML – INSERT Statements & Test Data

Each table is seeded with at least 10 rows of realistic test data. Data was chosen to exercise foreign-key references, multi-valued attributes, nullable columns, and the stock-decrement trigger.

6.1 Laboratorian (10 rows)

```
123 • INSERT INTO Laboratorian (Laboratorian_ID, Name, Street, City, Apartment) VALUES
124 (1, 'James Anderson', '123 Maple St', 'New York', 'Apt 3'),
125 (2, 'Emily Johnson', '456 Oak Ave', 'Los Angeles', 'Apt 7'),
126 (3, 'Michael Williams', '789 Pine Blvd', 'Chicago', 'Apt 12'),
127 (4, 'Sarah Davis', '321 Elm St', 'Houston', 'Apt 5'),
128 (5, 'Robert Martinez', '654 Cedar Rd', 'Phoenix', 'Apt 2'),
129 (6, 'Jessica Wilson', '987 Birch Lane', 'Philadelphia', 'Apt 9'),
130 (7, 'David Thompson', '147 Walnut Dr', 'San Antonio', 'Apt 1'),
131 (8, 'Ashley Garcia', '258 Spruce Ave', 'San Diego', 'Apt 6'),
132 (9, 'Christopher Lee', '369 Chestnut Blvd', 'Dallas', 'Apt 4'),
133 (10, 'Amanda White', '741 Willow Way', 'San Jose', 'Apt 8');
---
```

ID	Name	Street	City	Apartment
1	James Anderson	123 Maple St	New York	Apt 3
2	Emily Johnson	456 Oak Ave	Los Angeles	Apt 7
3	Michael Williams	789 Pine Blvd	Chicago	Apt 12
4	Sarah Davis	321 Elm St	Houston	Apt 5
5	Robert Martinez	654 Cedar Rd	Phoenix	Apt 2
6	Jessica Wilson	987 Birch Lane	Philadelphia	Apt 9
7	David Thompson	147 Walnut Dr	San Antonio	Apt 1
8	Ashley Garcia	258 Spruce Ave	San Diego	Apt 6
9	Christopher Lee	369 Chestnut Blvd	Dallas	Apt 4
10	Amanda White	741 Willow Way	San Jose	Apt 8

6.2 Lab_Phone (13 rows – multi-valued attribute)

```
135 • INSERT INTO Lab_Phone (Laboratorian_ID, Lab_phone) VALUES
136     (1, '212-555-0101'),
137     (2, '310-555-0102'),
138     (3, '312-555-0103'),
139     (4, '713-555-0104'),
140     (5, '602-555-0105'),
141     (6, '215-555-0106'),
142     (7, '210-555-0107'),
143     (8, '619-555-0108'),
144     (9, '214-555-0109'),
145     (10, '408-555-0110'),
146     (1, '212-555-0111'),
147     (3, '312-555-0112'),
148     (5, '602-555-0113');
```

Laboratorian_ID	Lab_phone
1	212-555-0101
2	310-555-0102
3	312-555-0103
4	713-555-0104
5	602-555-0105
6	215-555-0106
7	210-555-0107
8	619-555-0108
9	214-555-0109
10	408-555-0110
1	212-555-0111
3	312-555-0112
5	602-555-0113

6.3 COMPONENT (10 rows)

```
150 • INSERT INTO COMPONENT (Component_ID, Name_Of_Component, Available_quantity, Minimum_quantity) VALUES
151   ( 1, 'Test Tubes',      20, 100),
152   ( 2, 'Pipettes',       300, 50),
153   ( 3, 'Glucose Reagent', 15, 30),
154   ( 4, 'Blood Agar Plate', 200, 40),
155   ( 5, 'Urine Strips',    10, 80),
156   ( 6, 'Lancets',        600, 120),
157   ( 7, 'Centrifuge Tubes', 250, 50),
158   ( 8, 'Saline Solution', 5, 20),
159   ( 9, 'Microscope Slides', 350, 70),
160   (10, 'EDTA Tubes',     280, 60);
```

ID	Name	Available Qty	Min Qty
1	Test Tubes	20	100
2	Pipettes	300	50
3	Glucose Reagent	15	30
4	Blood Agar Plate	200	40
5	Urine Strips	10	80
6	Lancets	600	120
7	Centrifuge Tubes	250	50
8	Saline Solution	5	20
9	Microscope Slides	350	70
10	EDTA Tubes	280	60

6.4 MEDICAL_TEST (10 rows)

```
162 • INSERT INTO MEDICAL_TEST (Test_ID, Name_Of_Medical_Test, Related_components, Price, Laboratorian_ID) VALUE
163   ( 1, 'Blood Glucose', 'Glucose Reagent', 85, 1),
164   ( 2, 'Complete Blood', 'EDTA Tubes', 120, 2),
165   ( 3, 'Urinalysis', 'Urine Strips', 60, 3),
166   ( 4, 'Blood Culture', 'Blood Agar Plate', 200, 4),
167   ( 5, 'Lipid Panel', 'Centrifuge Tubes', 150, 5),
168   ( 6, 'Thyroid TSH', 'Test Tubes', 95, 6),
169   ( 7, 'HIV Screening', 'Lancets', 110, 7),
170   ( 8, 'Kidney Function', 'Saline Solution', 130, 8),
171   ( 9, 'Liver Panel', 'Pipettes', 140, 9),
172   (10, 'Malaria Test', 'Microscope Slides', 75, 10);
```

ID	Name	Related Components	Price (\$)	Lab ID
1	Blood Glucose	Glucose Reagent	85	1
2	Complete Blood	EDTA Tubes	120	2
3	Urinalysis	Urine Strips	60	3
4	Blood Culture	Blood Agar Plate	200	4
5	Lipid Panel	Centrifuge Tubes	150	5
6	Thyroid TSH	Test Tubes	95	6
7	HIV Screening	Lancets	110	7
8	Kidney Function	Saline Solution	130	8
9	Liver Panel	Pipettes	140	9
10	Malaria Test	Microscope Slides	75	10

6.5 ASSOCIATION (15 rows)

```

174 • INSERT INTO ASSOCIATION (Component_ID, Test_ID) VALUES
175     ( 3, 1),
176     (10, 2),
177     ( 1, 2),
178     ( 5, 3),
179     ( 4, 4),
180     ( 6, 4),
181     ( 7, 5),
182     ( 2, 5),
183     ( 1, 6),
184     ( 6, 7),
185     ( 8, 8),
186     ( 9, 9),
187     ( 2, 9),
188     ( 9, 10),
189     ( 6, 10);

```

Component_ID	Test_ID
3	1
10	2
1	2
5	3
4	4
6	4

7	5
2	5
1	6
6	7
8	8
9	9
2	9
9	10
6	10

6.6 PATIENT (10 rows)

```

191 • INSERT INTO Patient (Patient_ID, Patient_Name, Job, Birth_Date, Apartment, City, Street) VALUES
192 (12527, 'James Anderson', 'Engineer', '1985-03-12', 'Apt 5', 'New York', '5th Ave'),
193 (12528, 'Emily Johnson', 'Teacher', '1990-07-24', 'Apt 12', 'Los Angeles', 'Sunset Blvd'),
194 (12529, 'Michael Smith', 'Doctor', '1978-11-05', 'Apt 3', 'Chicago', 'Michigan Ave'),
195 (12530, 'Ashley Brown', 'Pharmacist', '1995-01-18', 'Apt 7', 'Houston', 'Main St'),
196 (12531, 'Chris Davis', 'Accountant', '1982-06-30', 'Apt 9', 'Phoenix', 'Central Ave'),
197 (12532, 'Jessica Wilson', 'Nurse', '1993-09-14', 'Apt 2', 'Philadelphia', 'Broad St'),
198 (12533, 'Daniel Martinez', 'Lawyer', '1975-04-22', 'Apt 15', 'San Antonio', 'Commerce St'),
199 (12534, 'Sarah Taylor', 'Architect', '1988-12-01', 'Apt 6', 'San Diego', 'Harbor Dr'),
200 (12535, 'Ryan Thomas', 'Software Dev', '1997-08-09', 'Apt 11', 'Dallas', 'Elm St'),
201 (12536, 'Lauren White', NULL, '2000-02-27', 'Apt 1', 'Austin', 'Congress Ave');

```

ID	Name	Job	Birth Date	City
12527	James Anderson	Engineer	1985-03-12	New York
12528	Emily Johnson	Teacher	1990-07-24	Los Angeles
12529	Michael Smith	Doctor	1978-11-05	Chicago
12530	Ashley Brown	Pharmacist	1995-01-18	Houston
12531	Chris Davis	Accountant	1982-06-30	Phoenix
12532	Jessica Wilson	Nurse	1993-09-14	Philadelphia
12533	Daniel Martinez	Lawyer	1975-04-22	San Antonio
12534	Sarah Taylor	Architect	1988-12-01	San Diego
12535	Ryan Thomas	Software Dev	1997-08-09	Dallas
12536	Lauren White	NULL	2000-02-27	Austin

6.7 PATIENT_PHONE (10 rows)

```
203 • INSERT INTO Patient_Phone (Patient_ID, Patient_Phone) VALUES
204     (12527, '2125550101'),
205     (12528, '3105550182'),
206     (12529, '3125550193'),
207     (12530, '7135550164'),
208     (12531, '6025550175'),
209     (12532, '2155550136'),
210     (12533, '2105550147'),
211     (12534, '6195550158'),
212     (12535, '2145550169'),
213     (12536, '5125550110');
```

Patient_ID	Patient_Phone
12527	2125550101
12528	3105550182
12529	3125550193
12530	7135550164
12531	6025550175
12532	2155550136
12533	2105550147
12534	6195550158
12535	2145550169
12536	5125550110

6.8 UNDERGOES (10 rows)

```
215 • INSERT INTO undergoes (Patient_ID, Test_ID) VALUES
216     (12527, 1),
217     (12528, 2),
218     (12529, 3),
219     (12530, 4),
220     (12531, 5),
221     (12532, 6),
222     (12533, 7),
223     (12534, 8),
224     (12535, 9),
225     (12536, 10);
```

Patient_ID	Test_ID
12527	1
12528	2
12529	3
12530	4
12531	5
12532	6
12533	7
12534	8
12535	9
12536	10

6.9 PREFORM (10 rows)

```

227 • INSERT INTO PREFORM (Laboratorian_ID, Test_ID) VALUES
228     (1, 1),
229     (2, 2),
230     (3, 3),
231     (4, 4),
232     (5, 5),
233     (6, 6),
234     (7, 7),
235     (8, 8),
236     (9, 9),
237     (10, 10);

```

Laboratorian_ID	Test_ID
1	1
2	2
3	3
4	4
5	5
6	6
7	7

8	8
9	9
10	10

6.10 TEST_RESULT (13 rows)

```

239 • INSERT INTO TEST_RESULT (Result, Laboratorian_ID, Patient_ID, TS_Date, Test_ID) VALUES
240 ('Negative', 1, 12527, '2025-06-10', 1),
241 ('Positive', 2, 12528, '2025-07-15', 2),
242 ('Normal', 3, 12529, '2025-08-20', 3),
243 ('Abnormal', 4, 12530, '2025-09-05', 4),
244 ('Negative', 5, 12531, '2025-10-10', 5),
245 ('Positive', 6, 12532, '2025-11-18', 6),
246 ('Normal', 7, 12533, '2025-12-01', 7),
247 ('Inconclusive', 8, 12534, '2026-01-12', 8),
248 ('Negative', 9, 12535, '2026-02-20', 9),
249 ('Positive', 10, 12536, '2026-03-02', 10),
250 ('Normal', 1, 12529, '2026-03-10', 3),
251 ('Abnormal', 2, 12531, '2026-04-15', 5),
252 ('Negative', 3, 12533, '2026-04-22', 7);

```

ID	Result	Lab ID	Patient ID	TS_Date	Test ID
1	Negative	1	12527	2025-06-10	1
2	Positive	2	12528	2025-07-15	2
3	Normal	3	12529	2025-08-20	3
4	Abnormal	4	12530	2025-09-05	4
5	Negative	5	12531	2025-10-10	5
6	Positive	6	12532	2025-11-18	6
7	Normal	7	12533	2025-12-01	7
8	Inconclusive	8	12534	2026-01-12	8
9	Negative	9	12535	2026-02-20	9
10	Positive	10	12536	2026-03-02	10
11	Normal	1	12529	2026-03-10	3
12	Abnormal	2	12531	2026-04-15	5
13	Negative	3	12533	2026-04-22	7

7. Views & Queries

7.1 Patient Test History (last 1 year) – Query 2

Retrieves all tests performed for patient 12527 within the past year, joined to test name and laboratorian name.

```
269 • -- Query 2: all tests performed by patient 12527 in the past year
270 SELECT
271     mt.Name_Of_Medical_Test,
272     tr.TS_Date,
273     l.Name AS Laboratorian_Name
274 FROM TEST_RESULT tr
275 JOIN MEDICAL_TEST mt ON tr.Test_ID = mt.Test_ID
276 JOIN Laboratorian l ON tr.Laboratorian_ID = l.Laboratorian_ID
277 WHERE tr.Patient_ID = 12527
278     AND tr.TS_Date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR);
```

7.2 Low-Stock Alert – Query 3

Returns all components whose available quantity has fallen below the defined minimum threshold. Critical for laboratory restocking operations.

```
284 -- Query 3: low-stock alert
285 • SELECT Component_ID, Name_Of_Component, Available_quantity, Minimum_quantity
286 FROM COMPONENT
287 WHERE Available_quantity < Minimum_quantity;
```

Expected output (seed data): Test Tubes (20 < 100), Glucose Reagent (15 < 30), Urine Strips (10 < 80), Saline Solution (5 < 20).

8. Index

A composite index on COMPONENT accelerates the low-stock alert query. Without it the query requires a full table scan on every alert check.

```
280      -- Index for low-stock alerts
281 • CREATE INDEX idx_low_stock
282      ON COMPONENT (Available_quantity, Minimum_quantity);
```

Index Name	Table	Columns	Purpose
idx_low_stock	COMPONENT	Available_quantity, Minimum_quantity	Speeds up low-stock WHERE clause
PRIMARY	All tables	Per-table PK columns	Auto-created unique clustered index
FK indexes	Tables with FKs	FK column(s)	Auto-created by MySQL for FK integrity

9. Trigger

A single AFTER INSERT trigger on TEST_RESULT automatically decrements Available_quantity of every COMPONENT associated with the newly inserted test, keeping inventory accurate without application-level logic.

```
254  -- Trigger
255  DELIMITER $$
256
257  • CREATE TRIGGER decrement_component_stock
258  AFTER INSERT ON TEST_RESULT
259  FOR EACH ROW
260  BEGIN
261      UPDATE COMPONENT c
262      JOIN ASSOCIATION a ON c.Component_ID = a.Component_ID
263      SET c.Available_quantity = c.Available_quantity - 1
264      WHERE a.Test_ID = NEW.Test_ID;
265  END$$
```

Attribute	Detail
Name	decrement_component_stock
Event	AFTER INSERT
Table	TEST_RESULT
Action	UPDATE COMPONENT via ASSOCIATION JOIN
Purpose	Decrement Available_quantity by 1 for each component linked to the test
Benefit	Automates inventory management; prevents stale stock counts
Limitation	Decrements by fixed 1; could be extended to accept a quantity parameter

10. Design Rationale & Implementation Notes

- **Multi-valued Attributes**

Lab_Phone and Patient_Phone are extracted into separate tables with composite primary keys, satisfying 1NF and allowing multiple phone numbers per person.

- **Derived Attribute**

Age is shown in the ER diagram as a derived attribute (dashed ellipse) but is NOT stored in the Patient table. It is computed at query time from Birth_Date using `TIMESTAMPDIFF(YEAR, Birth_Date, CURDATE())` to avoid data inconsistency.

- **M:N Resolution**

Three many-to-many relationships are resolved with dedicated junction tables: PREFORM (Laborarian–MedicalTest), undergoes (Patient–MedicalTest), and ASSOCIATION (Component–MedicalTest), following standard relational mapping rules.

- **Deferred Foreign Keys**

MEDICAL_TEST references Laborarian, but Laborarian is created after MEDICAL_TEST to avoid circular dependency at creation time. The FK is added via `ALTER TABLE` after both tables exist. The same pattern applies to ASSOCIATION.

- **ON DELETE / ON UPDATE Policies**

TEST_RESULT uses `RESTRICT` on Laborarian and Patient FKs (a result must not be orphaned). MEDICAL_TEST uses `SET NULL` on Laborarian_ID. Phone tables use `CASCADE` so deleting a person automatically removes their phone records.

- **AUTO_INCREMENT PK**

Test_Result_ID uses `AUTO_INCREMENT` so the database assigns a unique identifier for each result without application involvement.

- **Inventory Index**

idx_low_stock is a composite covering index on (Available_quantity, Minimum_quantity). MySQL can satisfy the `WHERE` clause comparison from the index alone without touching table data pages.

- **Trigger Safety**

The trigger fires `AFTER INSERT` to guarantee the row is committed before updating COMPONENT. The JOIN through ASSOCIATION means only linked components are decremented; tests with no associated components produce zero updates.

- **Related_components Column**

MEDICAL_TEST retains a Related_components VARCHAR for a human-readable summary. The authoritative machine-readable link is through the ASSOCIATION table. This is a deliberate denormalisation for display convenience.

- **Normalisation Level**

The schema is in 3NF: every non-key attribute is fully functionally dependent on the primary key with no transitive dependencies. The only exception is Related_components (see above), retained by design.

11. Application / UI Source Code

The application is built in Python using the Tkinter GUI toolkit and connects to the MySQL project database through the `mysql.connector` library. The codebase is split into eight modules, each with a single responsibility:

File	Role
<code>db.py</code>	Database connection factory
<code>styles.py</code>	Shared colour constants and reusable widget builders
<code>main.py</code>	Application entry point and dashboard
<code>patients.py</code>	Patients CRUD window
<code>laboratorians.py</code>	Laboratorians CRUD window
<code>components.py</code>	Components inventory window
<code>tests.py</code>	Medical tests catalogue window
<code>results.py</code>	Test results CRUD window
<code>reports.py</code>	Analytical queries window

11.1 Database Connection — `db.py`

This module contains a single function, `get_connection()`, which returns a live `mysql.connector` connection object pointed at the local project database. Every module imports and calls this function rather than hard-coding connection parameters, so the host, user, and database name only need to be changed in one place.

```
db.py > get_connection
1  import mysql.connector
2  def get_connection():
3      return mysql.connector.connect(
4          host="localhost",
5          user="root",
6          password="Habiba2007",
7          database="project"
8  )
```

11.2 Shared Styles & Widget Builders — `styles.py`

All visual consistency is managed centrally in `styles.py`. It defines the colour palette used throughout the application:

Constant	Hex Value	Purpose
BG	#f4f6fb	Window background
NAVY	#1b2a4a	Header and footer bars
BLUE	#3d5a99	Primary button and tile colour
LBLUE	#6b85c2	Hover state
ACCENT	#5b8dee	Highlight strip and selection
WHITE	#ffffff	Card and entry backgrounds
SUBTEXT	#6b7c99	Secondary labels

```
styles.py > page_footer
1  BG      = "#f4f6fb"
2  NAVY    = "#1b2a4a"
3  BLUE    = "#3d5a99"
4  LBLUE   = "#6b85c2"
5  ACCENT  = "#5b8dee"
6  WHITE   = "#ffffff"
7  SUBTEXT = "#6b7c99"
8  ENTRY_BG = "#eaf0fb"
9  ROW_ALT  = "#dce6f7"
10
```

Beyond the colour constants, `styles.py` exports five reusable builder functions that every page module calls:

- `apply_treeview_style()` — configures the `ttk.Treeview` theme once (alternating row colour, heading background, selected-row highlight).

```
11 def apply_treeview_style():
12     from tkinter import ttk
13     s = ttk.Style()
14     s.theme_use("clam")
15     s.configure("Treeview",
16                 background=WHITE, foreground=NAVY,
17                 fieldbackground=WHITE, rowheight=30,
18                 font=("Georgia", 10)
19     )
20     s.configure("Treeview.Heading",
21                 background=NAVY, foreground=WHITE,
22                 font=("Georgia", 10, "bold"), relief="flat"
23     )
24     s.map("Treeview", background=[("selected", ACCENT)])
25
```

- `page_header(win, icon, title, subtitle)` — renders the dark navy banner at the top of every window with an emoji icon, bold title, and coloured accent strip underneath.

```

26 def page_header(win, icon, title, subtitle):
27     import tkinter as tk
28     hdr = tk.Frame(win, bg=NAVY, height=60)
29     hdr.pack(fill="x")
30     hdr.pack_propagate(False)
31     tk.Label(hdr, text=f"{icon} {title}",
32             font=("Georgia", 14, "bold"), bg=NAVY, fg=WHITE
33             ).place(relx=0.02, rely=0.5, anchor="w")
34     tk.Frame(win, bg=ACCENT, height=4).pack(fill="x")
35     tk.Label(win, text=subtitle, font=("Georgia", 9, "italic"),
36             bg=BG, fg=SUBTEXT).pack(anchor="w", padx=20, pady=(8, 2))
37

```

- `make_form(win, fields)` — lays out a two-column label/entry grid inside a white card frame and returns a `{field_name: Entry}` dictionary so each page can read typed values by name.

```

38 def make_form(win, fields):
39     import tkinter as tk
40     form = tk.Frame(win, bg=WHITE, padx=20, pady=14,
41                     highlightbackground=ACCENT, highlightthickness=1)
42     form.pack(padx=20, pady=6, fill="x")
43     entries = {}
44     for i, f in enumerate(fields):
45         tk.Label(form, text=f, bg=WHITE, fg=SUBTEXT,
46                 font=("Georgia", 9), anchor="w").grid(
47             row=i//2, column=(i%2)*2, padx=(0,6), pady=5, sticky="w")
48         e = tk.Entry(form, bg=ENTRY_BG, fg=NAVY, insertbackground=NAVY,
49                     font=("Georgia", 10), relief="flat", bd=0,
50                     highlightthickness=2, highlightbackground=LBLUE,
51                     highlightcolor=ACCENT)
52         e.grid(row=i//2, column=(i%2)*2+1, padx=(0,18), pady=5, sticky="ew")
53         entries[f] = e
54     form.columnconfigure(1, weight=1)
55     form.columnconfigure(3, weight=1)
56     return entries
57

```

- `make_buttons(win, actions)` — renders a horizontal row of styled flat buttons from a list of (label, callback, colour) tuples.

```

58 def make_buttons(win, actions):
59     import tkinter as tk
60     bf = tk.Frame(win, bg=BG)
61     bf.pack(pady=6)
62     for text, cmd, color in actions:
63         tk.Button(bf, text=text, command=cmd,
64                 bg=color, fg=WHITE, font=("Georgia", 10, "bold"),
65                 relief="flat", bd=0, cursor="hand2",
66                 activebackground=LBLUE, activeforeground=WHITE,
67                 padx=14, pady=7).pack(side="left", padx=6)
68     return bf
69

```

- `make_tree(win, cols, widths)` — builds a `ttk.Treeview` with a vertical scrollbar and the column definitions passed in.

```

70 def make_tree(win, cols, widths=None):
71     import tkinter as tk
72     from tkinter import ttk
73     apply_treeview_style()
74     tf = tk.Frame(win, bg=BG)
75     tf.pack(padx=20, pady=4, fill="both", expand=True)
76     tree = ttk.Treeview(tf, columns=cols, show="headings")
77     for j, col in enumerate(cols):
78         tree.heading(col, text=col)
79         tree.column(col, width=(widths[j] if widths else 130))
80     sb = ttk.Scrollbar(tf, orient="vertical", command=tree.yview)
81     tree.configure(yscrollcommand=sb.set)
82     tree.pack(side="left", fill="both", expand=True)
83     sb.pack(side="right", fill="y")
84     return tree
85

```

- `page_footer(win)` — attaches a small navy strip at the bottom of every window with the system name.

```

86 def page_footer(win):
87     import tkinter as tk
88     f = tk.Frame(win, bg=NAVY, height=30)
89     f.pack(fill="x", side="bottom")
90     f.pack_propagate(False)
91     tk.Label(f, text="Medical Laboratory System",
92             font=("Georgia", 8), bg=NAVY, fg="█" "#8fa3cc"
93             ).place(relx=0.5, rely=0.5, anchor="center")

```

Because all pages use these builders, visual changes (font, colour, spacing) only need to be made in one place and propagate everywhere automatically.

11.3 Dashboard — `main.py`

`main.py` is the entry point. It creates the root `tk.Tk` window, maximises it, and builds the dashboard in three layers:

1. Header — a fixed-height navy frame with the system title centred.
2. Tile grid — a 2 x 3 `grid_frame` where each cell contains a styled tile. Each tile shows a large emoji icon in a white circle, a bold label, and a short subtitle. Clicking any tile calls the matching `open_*`() function from the corresponding module.
3. Footer — a narrow navy strip with the copyright notice.

The tiles also implement hover effects: <Enter> and <Leave> bindings lighten the tile background and swap the accent bar colour, giving the interface a responsive feel without any additional library.



11.4 CRUD Pages — Patients, Laboratorians, Components, Medical Tests, Test Results

Each of the five data-management pages follows an identical structure, so the pattern is explained once here using Patients (patients.py) as the example; the same logic applies to all others.

Page Structure

1. A `tk.Toplevel` window is opened (so the dashboard remains visible behind it).
2. `page_header()` renders the banner.
3. `make_form()` builds the input fields for the entity's attributes.
4. `make_tree()` builds the data table below the form.

```

patients.py > open_patients > load
1  import tkinter as tk
2  from tkinter import messagebox
3  from db import get_connection
4  from styles import (BG, NAVY, BLUE, ACCENT, WHITE,
5  | | | | | page_header, make_form, make_buttons, make_tree, page_footer)
6
7  def open_patients():
8      win = tk.Toplevel()
9      win.title("Patients")
10     win.geometry("820x600")
11     win.configure(bg=BG)
12
13     page_header(win, "👤", "Patients", "Add, view and remove patient records")
14
15     fields = ["Patient ID", "Name", "Job", "Birth Date (YYYY-MM-DD)",
16 | | | "Apartment", "City", "Street", "Phone"]
17     entries = make_form(win, fields)
18
19     tree = make_tree(win,
20 | ("ID", "Name", "Job", "Birth Date", "City", "Phone"),
21 | (70, 160, 110, 110, 110, 120))
22

```

5. Three local functions handle the three operations:

- `load()` — clears the tree, opens a connection, runs a SELECT query, and inserts every row into the tree with alternating row colours.

```

23  def load():
24      for r in tree.get_children(): tree.delete(r)
25      conn = get_connection(); cur = conn.cursor()
26      cur.execute("""
27          SELECT p.Patient_ID, p.Patient_Name, p.Job,
28          |      p.Birth_Date, p.City, pp.Patient_Phone
29          FROM Patient p
30          LEFT JOIN Patient_Phone pp ON p.Patient_ID = pp.Patient_ID
31          """)
32      for i, row in enumerate(cur.fetchall()):
33          tree.insert("", tk.END, values=row, tags=("alt" if i%2 else "",))
34      tree.tag_configure("alt", background=█ "#dce6f7")
35      conn.close()
36

```

- `add()` — reads values from the entries dictionary, validates that required fields are not empty, runs an INSERT statement, commits, closes the connection, shows a success message, and calls `load()` to refresh.

```

37  def add():
38      pid = entries["Patient ID"].get()
39      name = entries["Name"].get()
40      if not pid or not name:
41          messagebox.showwarning("Missing", "Patient ID and Name are required."); return
42      conn = get_connection(); cur = conn.cursor()
43      cur.execute(
44          "INSERT INTO Patient(Patient_ID, Patient_Name, Job, Birth_Date, Apartment, City, Street) VALUES(%s,%s,%s,%s,%s,%s,%s)",
45          (pid, name, entries["Job"].get(), entries["Birth Date (YYYY-MM-DD)"].get(),
46          entries["Apartment"].get(), entries["City"].get(), entries["Street"].get()))
47      ph = entries["Phone"].get()
48      if ph:
49          cur.execute("INSERT INTO Patient_Phone(Patient_ID, Patient_Phone) VALUES(%s,%s)", (pid, ph))
50      conn.commit(); conn.close()
51      messagebox.showinfo("Success", "Patient added!"); load()
52

```

- `delete()` — reads the selected row's primary key from `tree.item()`, runs a DELETE statement, commits, and refreshes.

6. `make_buttons()` wires those three functions to the Add, Delete, and Refresh buttons.
7. `page_footer()` attaches the footer.
8. `load()` is called immediately so the table is populated on first open.

```

53     def delete():
54         sel = tree.selection()
55         if not sel: messagebox.showwarning("Select", "Please select a patient."); return
56         pid = tree.item(sel[0])["values"][0]
57         conn = get_connection(); cur = conn.cursor()
58         cur.execute("DELETE FROM Patient_Phone WHERE Patient_ID=%s", (pid,))
59         cur.execute("DELETE FROM Patient WHERE Patient_ID=%s", (pid,))
60         conn.commit(); conn.close()
61         messagebox.showinfo("Deleted", "Patient deleted."); load()
62
63     make_buttons(win, [
64         ("⊕ Add Patient", add, BLUE),
65         ("🗑 Delete Selected", delete, "#8b2020"),
66         ("🔄 Refresh", load, "#4a5568"),
67     ])
68     page_footer(win)
69     load()

```

Variations Across Pages

While all five pages share the same structural template, each has minor differences to accommodate its data model:

Page	Notable difference
patients.py	Phone number inserted into separate Patient_Phone table; DELETE removes phone record first to satisfy FK constraint.
laboratorians.py	Same phone pattern with Lab_Phone. The load() query uses a LEFT JOIN to fetch the phone alongside the laboratorian row.
components.py	load() tags any row where Available_quantity < Minimum_quantity with a red background (#f7dcde) to flag low-stock items visually.
tests.py	No separate child table; the form includes Related Components, Price, and Laboratorian ID directly.
results.py	SELECT fetches six columns including FK IDs for Lab, Patient, and Test so the user sees all references at once.

11.5 Reports & Queries — reports.py

The Reports window exposes the two analytical queries as clickable buttons rather than requiring the user to type SQL.

How It Works

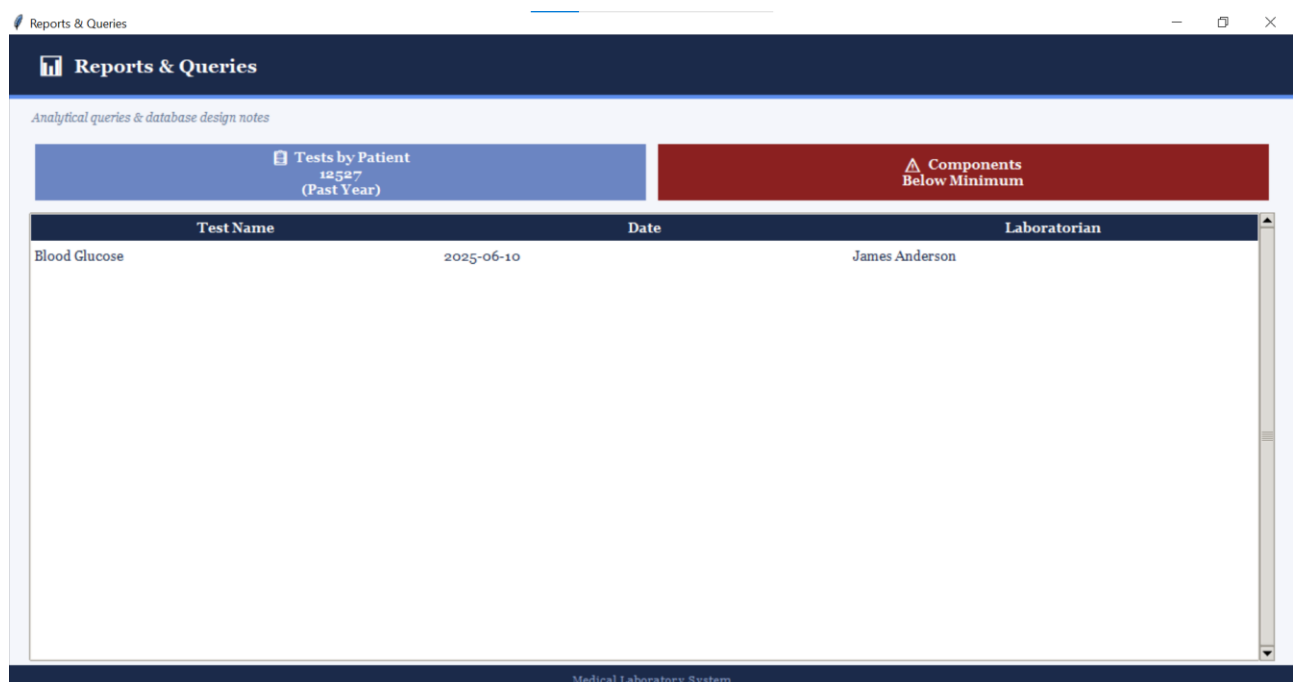
The window builds a single `tk.Treeview` with a generic Result column. Each query button calls a local `run(fn)` function that:

1. Calls the data function (e.g. `q1_data()`), which executes the SQL and returns a (column_names_tuple, rows_list) pair.
2. Clears the tree, rebuilds the column headings dynamically from the returned tuple, and inserts all rows with alternating colours.

This design means any new query can be added by writing one data function and one button entry — no structural change to the window is needed.

Query 1 — Tests by Patient 12527 (Past Year)

The function `q1_data()` joins `TEST_RESULT` with `MEDICAL_TEST` (to get the test name) and `Laboratorian` (to get the laboratorian's name), filtered to `Patient_ID = 12527` and `TS_Date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR)`. This is the same query described in Section 7.1.



The screenshot shows a window titled "Reports & Queries" with a dark blue header. Below the header, there are two buttons: "Tests by Patient 12527 (Past Year)" and "Components Below Minimum". Below the buttons is a table with three columns: "Test Name", "Date", and "Laboratorian". The table contains one row of data: "Blood Glucose", "2025-06-10", and "James Anderson".

Test Name	Date	Laboratorian
Blood Glucose	2025-06-10	James Anderson

Query 2 — Components Below Minimum

The function `q2_data()` selects all four columns from `COMPONENT` where `Available_quantity < Minimum_quantity`. This is the low-stock alert from Section 7.2. The result is displayed with the component ID, name, current quantity, and minimum threshold side by side so restocking decisions can be made immediately.



The screenshot shows a web application window titled "Reports & Queries". Inside, there's a header bar with the title and a sub-header "Analytical queries & database design notes". Below this, there are two tabs: "Tests by Patient 12527 (Past Year)" and "Components Below Minimum". The "Components Below Minimum" tab is active, displaying a table with four columns: "Component ID", "Component Name", "Available Qty", and "Minimum Qty". The table contains four rows of data, each representing a component that is below its minimum stock level.

Component ID	Component Name	Available Qty	Minimum Qty
1	Test Tubes	20	100
3	Glucose Reagent	15	30
5	Urine Strips	10	80
8	Saline Solution	5	20

Medical Laboratory System